

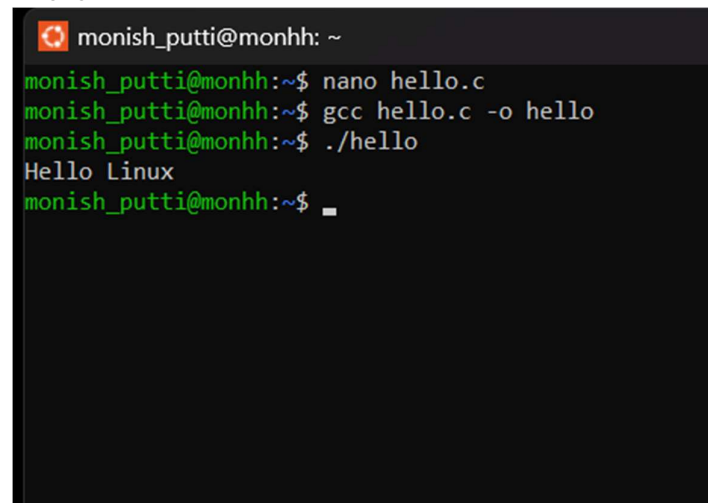
1.Hello.c

```
#include<stdio.h>
```

```
int main()
{
    printf("Hello Linux\n");
    return 0;
}
```

Compile: gcc hello.c -o hello

./hello

A terminal window with a dark background and light green text. The prompt is 'monish_putti@monhh: ~'. The user enters 'nano hello.c', then 'gcc hello.c -o hello', then './hello'. The output is 'Hello Linux'. The prompt returns to 'monish_putti@monhh: ~' with a cursor.

```
monish_putti@monhh: ~
monish_putti@monhh:~$ nano hello.c
monish_putti@monhh:~$ gcc hello.c -o hello
monish_putti@monhh:~$ ./hello
Hello Linux
monish_putti@monhh:~$
```

2.Read User Name

```
#include<stdio.h>
```

```
int main()
{
    char name[30];

    printf("Enter your name: ");
    scanf("%s",name);

    printf("Welcome %s\n",name);

    return 0;
}
```

```
monish_putti@monhh: ~  
Hello Linux  
monish_putti@monhh:~$ nano username.c  
monish_putti@monhh:~$ gcc username.c -o username  
monish_putti@monhh:~$ ./username  
Enter your name: Monish Putti  
Welcome Monish  
monish_putti@monhh:~$
```

3.Display Process ID

```
#include<stdio.h>  
#include<unistd.h>  
  
int main()  
{  
    printf("Process ID = %d\n",getpid());  
  
    return 0;  
}
```

```
monish_putti@monhh: ~  
monish_putti@monhh:~$ nano pid.c  
monish_putti@monhh:~$ gcc pid.c -o pid  
monish_putti@monhh:~$ ./pid  
Process ID = 627  
monish_putti@monhh:~$
```

4.Fork()

```
#include<stdio.h>  
#include<unistd.h>  
  
int main()  
{  
    fork();  
  
    printf("Hello\n");  
  
    return 0;  
}
```

```
monish_putti@monhh: ~  
Process ID = 627  
monish_putti@monhh:~$ nano fork.c  
monish_putti@monhh:~$ gcc fork.c -o fork  
monish_putti@monhh:~$ ./fork  
Hello  
Hello  
monish_putti@monhh:~$
```

5.Parent and Child

```
#include<stdio.h>  
#include<unistd.h>  
  
int main()  
{  
    pid_t pid;  
  
    pid=fork();  
  
    if(pid==0)  
    {  
        printf("I am Child Process\n");  
    }  
    else  
    {  
        printf("I am Parent Process\n");  
    }  
  
    return 0;  
}
```

```
monish_putti@monhh: ~  
monish_putti@monhh:~$ nano parent_child.c  
monish_putti@monhh:~$ gcc parent_child.c -o parent_child  
monish_putti@monhh:~$ ./parent_child  
I am Parent Process  
I am Child Process  
monish_putti@monhh:~$
```

6.Display parent and Child PID

```
#include<stdio.h>  
#include<unistd.h>  
  
int main()  
{  
    pid_t pid;
```

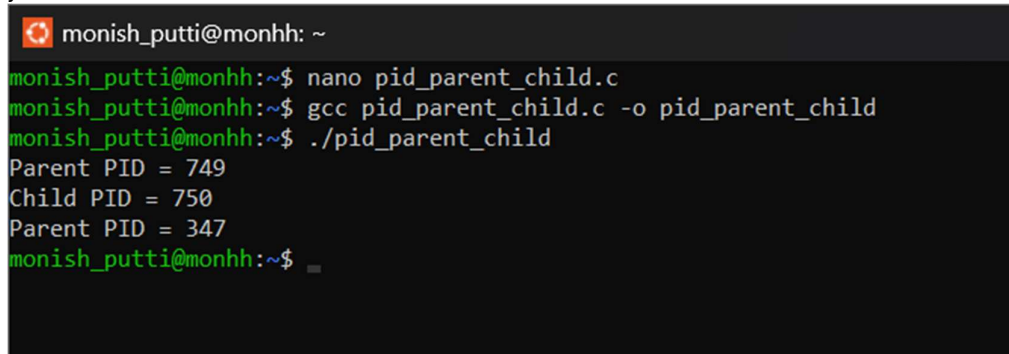
```

pid=fork();

if(pid==0)
{
    printf("Child PID = %d\n",getpid());
    printf("Parent PID = %d\n",getppid());
}
else
{
    printf("Parent PID = %d\n",getpid());
}

return 0;
}

```



```

monish_putti@monhh: ~
monish_putti@monhh:~$ nano pid_parent_child.c
monish_putti@monhh:~$ gcc pid_parent_child.c -o pid_parent_child
monish_putti@monhh:~$ ./pid_parent_child
Parent PID = 749
Child PID = 750
Parent PID = 347
monish_putti@monhh:~$

```

7.Wait()

```

#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("Child Running\n");
    }
    else
    {
        wait(NULL);

        printf("Parent Running After Child\n");
    }

    return 0;
}

```

```
Select monish_putti@monhh: ~
monish_putti@monhh:~$ nano wait.c
monish_putti@monhh:~$ gcc wait.c -o wait
monish_putti@monhh:~$ ./wait
Child Running
Parent Running After Child
monish_putti@monhh:~$
```

8.Wait()

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    printf("Before exec()\n");

    execlp("ls","ls",NULL);

    printf("After exec()\n");

    return 0;
}
```

```
monish_putti@monhh: ~
monish_putti@monhh:~$ nano exec.c
monish_putti@monhh:~$ gcc exec.c -o exec
monish_putti@monhh:~$ ./exec
Before exec()
Demo    file1    fork.c    hello.c    parent_child.c    pid_parent_child.c    shortcut    wait.c
OSLab   file1.txt  forkexec  moni       pid            pidparentchild        username
exec    file2     forkexec.c  moni.c     pid.c          pidparentchild.c      username.c
exec.c  fork      hello      parent_child  pid_parent_child  script.sh             wait
monish_putti@monhh:~$
```

9.fork() + exec()

```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

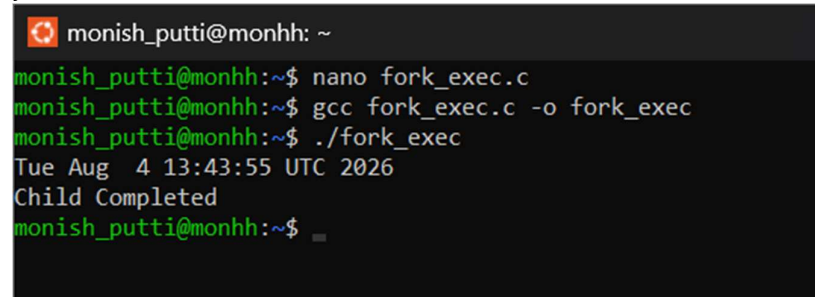
    pid=fork();

    if(pid==0)
    {
        execlp("date","date",NULL);
    }
}
```

```
else
{
    wait(NULL);

    printf("Child Completed\n");
}

return 0;
}
```



```
monish_putti@monhh: ~
monish_putti@monhh:~$ nano fork_exec.c
monish_putti@monhh:~$ gcc fork_exec.c -o fork_exec
monish_putti@monhh:~$ ./fork_exec
Tue Aug 4 13:43:55 UTC 2026
Child Completed
monish_putti@monhh:~$
```